

SECTION A - CORE CONCEPT MCQs

TOPIC 1: ARRAY CREATION, ATTRIBUTES & DTYPES (Q1–Q18)

- Q1.** What is the default dtype of `np.array([1, 2, 3])` on most 64-bit Linux/Mac systems?
a) int32 b) int64 c) float64 d) object
- Q2.** What is the dtype of `np.zeros((3,4))`?
a) int64 b) float64 c) bool d) object
- Q3.** What does `np.array([1, 2, "3"])` produce?
a) Array of ints [1,2,3] b) Array of strings ['1','2','3'] c) TypeError d) Array of floats
- Q4.** `range(0,10,2)` vs `np.arange(0,10,2)` - what is the main practical difference?
a) arange is always slower b) arange returns a NumPy array supporting vectorized math; range returns a plain Python range object c) They are identical in every way d) arange cannot use negative steps
- Q5.** What does `np.linspace(0, 10, 5)` return?
a) [0,2,4,6,8] b) [0,2.5,5,7.5,10] c) [0,2,4,6,8,10] d) [2,4,6,8,10]
- Q6.** What is the shape of `np.array([[1,2,3],[4,5,6]])`?
a) (2,3) b) (3,2) c) (6,) d) (2,3,1)
- Q7.** What is the dtype of `np.full((2,3), 7)` when 7 is a plain Python int?
a) float64 b) int64 c) object d) Always int32
- Q8.** How many bytes does a (1000, 1000) float64 array occupy?
a) 1,000,000 bytes b) 8,000,000 bytes c) 1,000,000 KB d) 4,000,000 bytes
- Q9.** Why might you deliberately choose `dtype=np.float32` over the default float64 for a large image-pixel array?
a) float32 is more precise b) float32 halves memory usage at the cost of precision - valuable for very large datasets like images c) float32 is required by matplotlib d) float32 enables GPU-only operations
- Q10.** What does `np.array([100, 100, 100], dtype=np.int8).sum()` return?
a) 44, due to int8 overflow b) Raises OverflowError c) 300 - NumPy upcasts the accumulator to the platform default integer type for narrow dtypes d) 300.0 as a float
- Q11.** What does `np.int8(100) + np.int8(100)` return?
a) 200 b) -56 - wraps around due to int8 overflow since the result stays in the narrow dtype c) Raises OverflowError d) 200.0
- Q12.** Which attribute returns the number of dimensions of an array?
a) .shape b) .ndim c) .size d) .dtype
- Q13.** For a (4,5) array, what does `.size` return?
a) 4 b) 5 c) 9 d) 20
- Q14.** What is the dtype of `np.array([True, False, True])`?

a) int b) bool c) object d) float

Q15. What does `np.eye(3)` produce?

a) A 3×3 array of all 1s b) A 3×3 identity matrix with 1s on the diagonal, 0s elsewhere c) A 1D array [0,1,2] d) A 3×3 array of random values

Q16. What is the purpose of `-1` in `np.arange(10).reshape(2, -1)`?

a) It removes the last column b) NumPy infers that dimension automatically so the total element count stays constant c) It reverses the array d) It raises an error

Q17. Which function creates an array filled with a single repeated constant value of your choosing?

a) `np.zeros()` b) `np.ones()` c) `np.full()` d) `np.eye()`

Q18. `np.array([[1,2],[3,4,5]])` - a jagged/ragged nested list - what happens in modern NumPy (≥1.24)?

a) Creates a 2×3 array padded with NaN b) Raises a `ValueError` because the sub-lists have inconsistent lengths and `dtype=object` is required explicitly c) Silently truncates to 2×2 d) Works fine and creates a (2,) object array automatically

TOPIC 2: INDEXING, SLICING & VIEWS VS COPIES (Q19–Q36)

Q19. `a = np.array([10,20,30,40,50]); a[1:3]` returns?

a) [20,30] b) [20,30,40] c) [10,20,30] d) [30,40]

Q20. `a[-1]` for `a=np.array([1,2,3])` returns?

a) 1 b) 3 c) Error d) None

Q21. `m[1,2]` for `m=np.arange(12).reshape(3,4)` returns?

a) 5 b) 6 c) 4 d) 7

Q22. Does `a[1:4] = some_values` (slice assignment) create a copy or modify in place?

a) Creates a new array b) Modifies the original array's underlying memory directly, in place c) Raises an error d) Only works for 1D arrays

Q23. `view = original[2:5]; view[0] = 99` - what happens to `original`?

a) Unchanged - slicing always copies b) `original[2]` becomes 99 - slicing returns a view sharing memory c) Raises an error d) Only `original.dtype` changes

Q24. How do you force an independent copy when slicing an array?

a) Use `.view()` b) Use `.copy()` c) Slicing is already independent by default d) Use `np.array(arr, view=False)`

Q25. `arr[arr > 5]` (boolean mask indexing) - does modifying the result affect the original array?

a) Yes - boolean masking returns a view b) No - boolean masking always returns a NEW independent array (a copy) c) Only for 1D arrays d) Only if dtype matches

Q26. `arr[[0,2,4]]` (fancy/integer-array indexing) - view or copy?

a) View - shares memory b) Copy - an independent array c) Depends on array size d) Raises an error

Q27. `flat.reshape(3,4)` - view or copy of `flat`?

a) A view whenever possible - shares memory b) Always a copy c) A view only if flat is 1D d) A copy only if dtype changes

Q28. `m[:, 1]` extracts what from a 2D array `m`?

a) Row 1 b) Column 1 c) A single element d) The diagonal

Q29. `m[::2, :]` selects what?

a) Every other column b) Every other row, all columns c) Only the first row d) Only the last row

Q30. What does `np.shares_memory(a, b)` check?

a) Whether a and b have equal values b) Whether a and b refer to overlapping underlying memory buffers c) Whether a and b have the same shape d) Whether a and b have the same dtype

Q31. `a = np.arange(10); b = a[::2]; b[:] = 0` - what happens to `a`?

a) a is unaffected b) Every other element of a becomes 0 - `a[::2]` is a view c) Raises an error d) a becomes entirely 0

Q32. Why is `df.copy()` (a Pandas convention you will see soon) directly inspired by NumPy's view/copy distinction?

a) It is unrelated to NumPy b) Pandas DataFrames are built on NumPy arrays, and slicing a DataFrame can return a view; `.copy()` guarantees independence, preventing SettingWithCopyWarning bugs c) `.copy()` makes code run faster d) `.copy()` changes the dtype

Q33. `m[0:2, 2:4]` on a (3,4) matrix extracts what shape?

a) (2,4) b) (2,2) c) (3,2) d) (1,2)

Q34. What does negative-step slicing `a[::-1]` do?

a) Returns an empty array b) Reverses the array c) Returns every other element d) Raises an error

Q35. `a[2:]` versus `a[2:len(a)]` - is there any practical difference?

a) `a[2:]` raises an error b) They are functionally identical - omitting the stop means "to the end" c) `a[2:len(a)]` is faster d) They return different dtypes

Q36. A junior analyst writes `subset = data[data["flag"]==1]; subset["value"] = 0` and is confused why the original `data` is untouched. Why?

a) Boolean-mask indexing always returns a copy, so modifying `subset` does not propagate back to `data` b) This is a NumPy bug c) The mask was applied incorrectly d) `subset` and `data` always share memory regardless

TOPIC 3: UFUNCS, ELEMENT-WISE OPERATIONS & DTYPE PROMOTION (Q37–Q54)

Q37. `np.array([1,2,3]) + np.array([10,20,30])` returns?

a) [1,2,3,10,20,30] b) [11,22,33] c) Error - shapes incompatible d) [10,40,90]

Q38. `np.sqrt(np.array([4,9,16], dtype=int))` - what is the dtype of the result?

a) int64 - same as input b) float64 - sqrt always returns floating-point results, even for integer input c) object d) bool

Q39. `np.exp(np.array([0,1]))` returns approximately?

a) [0, 1] b) [1.0, 2.718...] c) [1, e] d) [0, e]

Q40. `np.array([1,2,3], dtype=np.int32) + np.array([1.5,2.5,3.5])` - resulting dtype?`

a) int32 b) float64 c) object d) Raises TypeError

Q41. `np.clip(np.array([-5, 3, 15, 8]), 0, 10)` returns?`

a) [-5,3,15,8] b) [0,3,10,8] c) [0,0,0,0] d) [10,10,10,10]

Q42. `np.abs(np.array([-3,-1,2,-7]))` returns?`

a) [3,1,2,7] b) [-3,-1,2,-7] c) [3,1,-2,7] d) Error

Q43. Which ufunc computes the element-wise NATURAL logarithm?

a) `np.log10()` b) `np.log()` c) `np.ln()` d) `np.exp()`

Q44. `np.floor(-2.3)` vs np.trunc(-2.3)` - do they give the same result?`

a) Yes, always identical b) No - floor rounds toward negative infinity (-3), trunc rounds toward zero (-2) c) floor raises an error for negative numbers d) trunc only works on positive numbers

Q45. `np.array([-7]) // 2` returns?`

a) -3 b) -4 - floor division rounds toward negative infinity, not toward zero c) -3.5 d) 3

Q46. `np.sign(np.array([-4, 0, 9]))` returns?`

a) [-1, 0, 1] b) [-4, 0, 9] c) [-1, 1, 1] d) Error - sign is undefined for 0

Q47. What is the key difference between plain Python `1/0` and np.array([1])/np.array([0])`?`

a) Both raise ZeroDivisionError b) Python raises ZeroDivisionError; NumPy returns inf (with a RuntimeWarning) instead of crashing c) Both silently return 0 d) Both return NaN

Q48. `np.array([1,2,0]) / np.array([0,2,0])` returns?`

a) [inf, 1.0, nan] b) Raises an error c) [0, 1, 0] d) [nan, 1.0, inf]

Q49. Which operator performs TRUE (float) division in Python 3 / NumPy?

a) `//` b) `/` c) `%` d) `**`

Q50. `np.multiply(a, b)` vs a * b` for two arrays - functionally?`

a) Different results b) Identical - `**` is simply operator-overloading syntax for np.multiply() c) np.multiply is deprecated d) **` only works for scalars`

Q51. Ufuncs like `np.sqrt` and `np.sin` process arrays how, internally?

a) One Python-loop iteration per element b) Element-wise, in compiled C, applied to the entire array at once c) Only on the first element d) Row-by-row only, for 2D arrays

Q52. `np.round(2.5)` and np.round(3.5)` - what do they return?`

a) 3 and 4 b) 2 and 4 - NumPy uses round-half-to-even (banker's rounding), not round-half-up c) 3 and 4 always d) 2 and 3

Q53. Which statement about ufunc broadcasting with a scalar is correct?

a) Scalars cannot be combined with arrays b) A scalar is treated as if it were broadcast to match the array's entire shape c) Scalars always convert the array to object dtype d) Scalars only work with addition

Q54. A pipeline computes `np.log(data)` where some values in data` are zero or negative. What happens, and what is the safe fix?`

a) Raises a fatal error, stopping the program b) Returns -inf for log(0) and NaN for log(negative), with a RuntimeWarning, WITHOUT stopping execution - fix by clipping data to a small positive floor first, e.g. ``np.log(np.clip(data, 1e-10, None))`` c) Automatically skips invalid values d) Silently returns 0 for invalid inputs

TOPIC 4: AGGREGATION, AXIS & STATISTICAL FUNCTIONS (Q55–Q72)

Q55. For ``m=np.arange(12).reshape(3,4)``, what does ``m.sum()`` (no axis) return?

a) [6,22,38] b) 66 - a single scalar c) [12,15,18,21] d) Error

Q56. ``m.sum(axis=0)`` collapses which dimension?

a) Columns, giving one value per row b) Rows, giving one value per column c) Nothing changes
d) Both axes

Q57. ``m.sum(axis=1)`` collapses which dimension?

a) Rows, giving one value per column b) Columns, giving one value per row c) Nothing changes
d) Both axes

Q58. For a (3,4) array, what is the shape of ``m.mean(axis=0)``?

a) (3,) b) (4,) c) (3,4) d) scalar

Q59. For a (3,4) array, what is the shape of ``m.mean(axis=1)``?

a) (3,) b) (4,) c) (3,4) d) scalar

Q60. What does ``axis=-1`` mean in an aggregation call?

a) Aggregates over no axis at all b) Refers to the LAST axis of the array, regardless of its total number of dimensions c) Always means axis=0 d) Raises an error

Q61. What is the default ``ddof`` (delta degrees of freedom) for ``np.std()``?

a) 1 - sample standard deviation b) 0 - population standard deviation c) N - the number of elements d) Depends on array size

Q62. ``np.array([2,4,4,4,5,5,7,9]).std()`` (default ``ddof=0``) is approximately?

a) 2.0 b) 2.14 c) 4.0 d) 1.0

Q63. Which function gives the INDEX of the maximum value in an array, not the value itself?

a) `np.max()` b) `np.argmax()` c) `np.amax()` d) `np.maxindex()`

Q64. ``data.mean(axis=1, keepdims=True)`` for a (3,4) array - what is the resulting shape?

a) (3,) b) (3,1) c) (1,4) d) (3,4)

Q65. ``np.cumsum([1,2,3,4])`` returns?

a) [1,2,3,4] b) [1,3,6,10] c) [10,10,10,10] d) [4,3,2,1]

Q66. What does ``np.percentile(data, 50)`` equal?

a) The mean b) The median c) The mode d) The maximum

Q67. ``np.percentile(data, [25,75])`` is commonly used to compute what?

a) The standard deviation b) The interquartile range (IQR) bounds, Q1 and Q3 c) The min and max d) The mode

Q68. What is the correct mental model for axis-based aggregation?

a) axis=0 always means "horizontal" b) "axis=N" names the dimension that DISAPPEARS/collapses after the operation c) axis has no effect on the result shape d) axis=1 always returns a scalar

Q69. Why does `np.mean()` on an integer array always return a float, even though `np.sum()` on the same array can return an integer?

a) It is inconsistent / a bug b) Division (required for mean) inherently produces non-integer results in general, so NumPy always promotes the output to float64; `sum()` has no such requirement and can keep the integer dtype c) `mean()` is implemented in pure Python, `sum()` in C d) Both always return float

Q70. `np.corrcoef(x, y)` returns what structure?

a) A single correlation coefficient b) A 2×2 correlation matrix whose off-diagonal entries are the Pearson r between x and y c) A list of p-values d) The covariance only

Q71. How does `np.median()` differ from `np.mean()` in terms of outlier sensitivity?

a) Median is more sensitive to outliers b) Median is robust to outliers (a positional measure); mean is pulled toward extreme values c) They behave identically for any data d) Median only works on already-sorted arrays

Q72. For a (40, 6) array `scores` representing (students × subjects), what does `scores.mean(axis=0).shape` represent?

a) (40,) - average score per student b) (6,) - average score per subject, across all 40 students c) (40,6) - unchanged d) scalar - overall average

TOPIC 5: BROADCASTING (Q73–Q90)

Q73. Can you add a scalar directly to an array, e.g. `np.array([1,2,3]) + 10`?

a) No, raises `TypeError` b) Yes - the scalar broadcasts to every element c) Only for float arrays d) Only works with `np.add()`

Q74. Shapes (3,4) and (4,) - do they broadcast together, and what is the result shape?

a) No, incompatible b) Yes - (4,) is treated as (1,4) and stretched across rows; result shape (3,4) c) Yes, but result is (4,3) d) Yes, result is (3,4,4)

Q75. Shapes (3,4) and (3,) - do they broadcast?

a) Yes, automatically b) No - (3,) is treated as (1,3), which does not match the trailing dimension 4 → `ValueError` c) Yes, result is (3,4) d) Yes, result is (3,3)

Q76. Shapes (3,1) and (1,4) - what is the broadcast result shape?

a) Error - incompatible b) (3,4) - both size-1 dimensions stretch to match the other array c) (1,1) d) (3,1)

Q77. What are the two core broadcasting rules?

a) Shapes must always be identical b) Compare shapes from the trailing (rightmost) dimension; dimensions are compatible if they are equal OR one of them is 1 c) Only 1D arrays can broadcast d) Broadcasting always requires explicit `reshape()` calls

Q78. Why does broadcasting NOT actually duplicate data in memory for the stretched dimension?

a) It does duplicate data - that is the whole point b) NumPy uses stride tricks to logically "view" the smaller array as repeated, without physically copying it - saving memory and time c) Broadcasting always raises MemoryError for large arrays d) Broadcasting only works for 1×1 arrays

Q79. `m - m.mean(axis=0)` for a (5,4) array `m` - what is this computing?

a) Row-wise centering b) Column-wise centering - subtracting each column's mean from every value in that column c) An error - shape mismatch d) The overall mean

Q80. Computing pairwise differences between all elements of a 1D array of length N (an N×N difference matrix) using broadcasting alone - which expression does it?

a) `x - x` b) `x[:, np.newaxis] - x[np.newaxis, :]` c) `x.reshape(N,N) - x` d) `np.subtract(x, x, axis=0)`

Q81. `np.array([1,2,3]) * 2.5` - what is the result dtype?

a) int b) float64 - multiplying an int array by a float scalar upcasts the result c) Error - type mismatch d) bool

Q82. Shapes (8,1) and (1,6) broadcast together - what is the resulting shape, and what does this pattern usually represent?

a) Error - cannot combine two non-matching shapes b) (8,6) - a common pattern for building an "outer" pairwise grid (e.g. distance matrix, attention scores) without explicit loops c) (1,1) d) (48,) flattened

Q83. A (3,4,5) array and a (4,5) array - do they broadcast? What rule is applied?

a) No, ndim must match exactly b) Yes - the smaller array is treated as (1,4,5) and broadcasts across the first dimension c) Only if reshaped manually first d) Result has 3 fewer dimensions

Q84. When NumPy cannot broadcast two shapes, what exception is raised?

a) TypeError b) ValueError, describing the incompatible operand shapes c) A custom BroadcastError d) IndexError

Q85. `weights` has shape (3,4); `inputs` has shape (4,). What does `weights * inputs` compute (note: NOT matrix multiplication)?

a) Matrix multiplication, shape (3,) b) Element-wise multiplication, broadcasting inputs across each of the 3 rows, shape (3,4) c) Error - shape mismatch d) Dot product, a scalar result

Q86. `data - data.mean(axis=0)` for `data.shape == (50, 4)` - what shape must

`data.mean(axis=0)` have to broadcast successfully?

a) (50,4) b) (4,) - broadcasts automatically against the trailing dimension of data c) It will always raise an error d) Requires a manual reshape every time

Q87. When arrays have different `ndim`, on which side does NumPy pad the smaller shape with 1s?

a) On the right (trailing) b) On the left (leading) c) Both sides equally d) NumPy never pads automatically

Q88. `np.ones((5,3)) + np.ones((5,))` - does this broadcast? Why or why not?

a) Yes - (5,) becomes (1,5), and since trailing dims compare 3 vs 5, neither is 1 → this actually raises an error b) Yes, automatically gives shape (5,3) c) No issue, this always works d) Works only with explicit casting

Q89. What shape would ``np.ones((5,3)) + np.ones((5,1))`` broadcast to?

- a) Error b) (5,3) - the (5,1) array's size-1 column stretches across all 3 columns c) (5,1) d) (10,3)

Q90. A function computes ``z = (data - mean) / std`` where ``data.shape=(100,5)``, ``mean.shape=(5,)``, ``std.shape=(5,)``. Why does this z-score pattern "just work" with no explicit loop?

- a) It does NOT broadcast - shapes must match exactly b) It broadcasts correctly: (5,) aligns with the trailing dimension of (100,5), so each of the 5 per-column means/stds is subtracted/divided across all 100 rows simultaneously c) Only works if mean and std are scalars d) Raises a shape error because $100 \neq 5$

TOPIC 6: MATRIX OPERATIONS, RESHAPING & STACKING (Q91–Q108)

Q91. Which operator performs TRUE matrix multiplication (not element-wise) in modern NumPy?

- a) `*` b) `@` c) `%` d) `//`

Q92. For `A.shape=(3,4)` and `B.shape=(4,2)`, what is the shape of ``A @ B``?

- a) (3,2) b) (4,4) c) (3,4) d) Error - incompatible shapes

Q93. For `A.shape=(3,4)` and `B.shape=(3,4)`, is ``A @ B`` valid?

- a) Yes, shape (3,4) b) No - inner dimensions (4 and 3) do not match for matrix multiplication; use `A @ B.T` instead c) Yes, shape (4,4) d) Yes, an element-wise result

Q94. What operation does ``A.T`` perform?

- a) Inverts the matrix b) Transposes the matrix - swaps rows and columns c) Sorts the matrix d) Flattens the matrix

Q95. For `A.shape=(4,3)`, what is ``A.T.shape``?

- a) (4,3) b) (3,4) c) (12,) d) (3,3)

Q96. For `A.shape=(4,3)` and `v.shape=(3,)`, what is the shape of ``A @ v``?

- a) (4,) b) (3,) c) (4,3) d) Error

Q97. For `A.shape=(4,3)`, what is the shape of ``A.T @ A``?

- a) (4,4) b) (3,3) c) (4,3) d) (12,12)

Q98. ``flat.reshape(2,-1)`` for a 12-element array - what shape results?

- a) (2,6) b) (6,2) c) (2,12) d) Error

Q99. Can you reshape a 13-element array into shape (3,4)? Why or why not?

- a) Yes, with padding b) No - the total element count (13) must exactly equal the product of the new shape ($3 \times 4 = 12$); raises `ValueError` c) Yes, truncating the last element d) Yes, NumPy pads with zeros automatically

Q100. `.ravel()` vs `.flatten()` - what is the key practical difference?

- a) No difference at all b) `ravel()` returns a view when possible (shares memory); `flatten()` always returns an independent copy c) `flatten()` is a view; `ravel()` is a copy d) `ravel()` only works on 2D arrays

Q101. ``np.vstack([a, b])`` for two (2,3) arrays - what is the resulting shape?

a) (2,6) b) (4,3) c) (2,3,2) d) (3,4)

Q102. `np.hstack([a, b])` for two (2,3) arrays - what is the resulting shape?

a) (4,3) b) (2,6) c) (2,3,2) d) (6,2)

Q103. `np.concatenate([m1, m2])` with no axis argument - what is the default axis used?

a) axis=1 b) axis=0 c) axis=-1 d) Concatenates as flattened arrays

Q104. `np.concatenate([np.ones((3,4)), np.ones((2,5))], axis=0)` - what happens?

a) Works fine, gives shape (5,4) b) Raises ValueError - all dimensions except the concatenation axis must match exactly (4≠5) c) Works fine, gives shape (5,5) d) Pads automatically

Q105. `np.linalg.norm(v)` for a 1D vector - what does it compute by default?

a) The sum of elements b) The Euclidean (L2) norm - the square root of the sum of squared elements c) The maximum element d) The mean

Q106. Computing cosine similarity between rows of a matrix typically involves which two operations in sequence?

a) Transpose then sort b) Normalize each row to unit length (divide by its L2 norm), then matrix-multiply the normalized matrix by its own transpose c) Sum then divide by count d) reshape then concatenate

Q107. `np.identity(4)` vs `np.eye(4)` - practical difference for a plain square identity matrix?

a) identity() is faster b) For a plain square identity, they are functionally equivalent; eye() additionally supports offsetting the diagonal (k parameter) and non-square shapes c) identity() returns floats, eye() returns ints d) They are unrelated functions

Q108. For `A.shape=(1000,1000)`, `A @ A` and `A * A` differ in both RESULT and computational cost. Explain the difference.

a) Identical result, only speed differs b) `A @ A` is matrix multiplication (uses optimized BLAS, conceptually $O(n^3)$) producing a genuinely different mathematical result than `A * A`, which is element-wise ($O(n^2)$) and simply squares each entry independently c) `A * A` is undefined for square matrices d) Both always error for non-square inputs

TOPIC 7: BOOLEAN/FANCY INDEXING, SORTING, UNIQUE & RANDOM (Q109–Q126)

Q109. `a[a > 5]` returns what?

a) Indices where the condition is True b) The actual elements where the condition is True, as a new 1D array c) A boolean array only d) Raises an error

Q110. `(a > 5).sum()` computes what?

a) The sum of all elements greater than 5 b) The COUNT of elements greater than 5 (True counts as 1, False as 0) c) Always 0 d) Raises an error

Q111. Compound boolean condition `(a > 2) & (a < 8)` - why `&` instead of Python's `and`?

a) `and` is faster b) Python's `and`/`or` do not work element-wise on arrays (they expect a single boolean); `&`/`|` are NumPy's element-wise operators c) `&` only works on integers d) No real difference, purely stylistic

Q112. `a[a>5] = 0` - is this a view-based in-place modification, or does it create a copy?

a) Creates a copy, original untouched b) Modifies `a` IN PLACE - the boolean mask appears on the LEFT side of the assignment, writing directly into the original array c) Raises an error d) Only works for float arrays

Q113. `np.sort(a)` vs `a.sort()` - key difference?

a) Identical in every way b) `np.sort(a)` returns a NEW sorted array, leaving `a` unchanged; `a.sort()` sorts `a` IN PLACE and returns None c) `a.sort()` is always faster d) `np.sort()` only works on 1D arrays

Q114. What does `np.argsort(a)` return?

a) The sorted values b) The INDICES that would sort the array - not the values themselves c) The rank of each value d) A boolean mask

Q115. Given `scores=[88,55,94]` and `names=['Ahmad','Sara','Bilal']`, how do you get names ranked by score, descending?

a) `sorted(names)` b) `names[np.argsort(scores)[::-1]]` c) `np.sort(names)` d) `names[scores]`

Q116. `np.unique(a)` returns what?

a) Only the first element b) The sorted distinct values in `a` c) A count of all elements d) The original array unchanged

Q117. `np.unique(a, return\_counts=True)` returns what structure?

a) Just the unique values b) A tuple: (unique values array, frequency counts array) - two parallel arrays c) A dict d) A single combined array

Q118. `np.bincount()` works on what kind of input, and what is its main limitation versus `np.unique()`?

a) Any dtype, no limitation b) Non-negative INTEGERS only - it cannot be used directly on floats or strings, unlike np.unique() c) Strings only d) It is identical to np.unique()

Q119. What does `np.random.seed(42)` do?

a) Limits random values to 42 b) Makes the random number generator's output reproducible across runs c) Disables randomness entirely d) Resets all NumPy settings

Q120. `np.random.choice(arr, size=4, replace=False)` does what?

a) Picks 4 elements with possible repeats b) Picks 4 DISTINCT elements without repetition (errors if size exceeds len(arr)) c) Shuffles the entire array d) Always returns the same 4 elements

Q121. `np.random.shuffle(arr)` - does it return a new array or modify in place?

a) Returns a new shuffled array, `arr` unchanged b) Shuffles `arr` IN PLACE and returns None c) Raises an error for 1D arrays d) Only works for integer arrays

Q122. Two researchers run a simulation. One uses `np.random.seed(42)`; the other claims to "use the same code" but gets DIFFERENT random numbers. Most likely cause?

a) NumPy's random generator is fundamentally non-deterministic and cannot be seeded b) One of them forgot to set the seed, called other random functions before the simulation (shifting the generator's internal state), or used a different NumPy version/generator algorithm c) Seeds only work on Linux d) Seeding is purely cosmetic and has no real effect

Q123. `fancy = a[[0,2,4]]; fancy[0] = 999` - does `a[0]` change?

a) Yes - fancy indexing returns a view b) No - fancy (integer array) indexing always returns an independent copy c) Only if a is 2D d) Raises an error

Q124. What does `np.where(condition)` - a SINGLE argument, no x/y - return?

a) The filtered values directly b) A tuple of arrays giving the INDICES where the condition is True (one array per dimension) c) A single boolean d) Raises a TypeError - np.where always requires 3 arguments

Q125. `np.argmax(a)` returns what when there are TIES for the maximum value?

a) All tied indices as a list b) The index of the FIRST occurrence of the maximum value c) Raises an error d) A randomly chosen tied index

Q126. An analyst writes `top3_idx = np.argsort(scores)[:3]` intending to get the top 3 HIGHEST scores, but gets the 3 LOWEST instead. What is the bug, and the fix?

a) argsort() is broken b) argsort() sorts ASCENDING by default, so `[:3]` grabs the smallest values' indices; fix by reversing first: `np.argsort(scores)[::-1][:3]`, or equivalently `np.argsort(-scores)[:3]` c) Need to add `.reverse()` to scores first d) argsort() does not support slicing

ANSWER KEY

Q#	Ans	Tag	Key Concept
1	B		NumPy infers the platform default integer type - int64 on most 64-bit Linux/Mac systems (Windows can default to int32).
2	B		zeros(), ones(), and full() default to float64 unless dtype is explicitly specified.
3	B		NumPy arrays hold a single dtype; mixing types upcasts everything to the most general type - here, all elements become strings.
4	B		np.arange() materialises a NumPy array that participates in vectorized operations; range() is a lightweight Python iterator with no array math support.
5	B		linspace divides the interval into n evenly spaced points INCLUDING both endpoints: $10/(5-1)=2.5$ step size.
6	A		2 rows, 3 columns → shape (2,3).
7	B		full() infers the dtype from the fill value; an integer fill value produces an integer array using the platform

			default int type.
8	B		1,000,000 elements × 8 bytes/float64 = 8,000,000 bytes (~8 MB). nbytes = size × itemsize.
9	B		float32 uses 4 bytes instead of 8, halving memory and often improving cache performance, at the cost of reduced numerical precision.
10	C		For reduction operations like .sum(), NumPy automatically promotes integer dtypes narrower than the platform default (e.g. int8→int64) to protect the accumulator from overflow. Verified: a.sum() returns int64 dtype even though a.dtype is int8.
11	B		Unlike reduction methods (.sum()), direct scalar/elementwise arithmetic between two same-dtype values does NOT promote - the result silently wraps via two's-complement: 200 mod 256, interpreted as signed int8, equals -56.
12	B		.ndim returns an integer count of axes; .shape returns the tuple of sizes per axis.
13	D		.size is the TOTAL element count = product of all shape dimensions =

			$4 \times 5 = 20$.
14	B		NumPy infers a dedicated boolean dtype from Python bool literals.
15	B		<code>np.eye(n)</code> is the standard way to build an $n \times n$ identity matrix.
16	B		-1 tells NumPy to compute the missing dimension from the total size and the other given dimensions.
17	C		<code>np.full(shape, value)</code> is the dedicated constant-fill function; <code>zeros/ones</code> are special cases of <code>full()</code> for 0 and 1.
18	B		Since NumPy 1.24, creating a ragged array without explicitly passing <code>dtype=object</code> raises a <code>ValueError</code> instead of silently falling back to an object array.
19	A		Slicing <code>[1:3]</code> takes indices 1 and 2 (stop is exclusive): values 20 and 30.
20	B		Negative indexing counts from the end; -1 is the last element.
21	B		Row 1 is <code>[4,5,6,7]</code> ; index 2 within that row is 6.
22	B		Slice assignment writes the new values directly into the existing memory

			buffer.
23	B		Basic slicing (start:stop:step) returns a VIEW into the same underlying buffer; modifying the view modifies the source array.
24	B		.copy() explicitly allocates new memory and duplicates the data, breaking the shared- memory link.
25	B		Boolean indexing on the right-hand side always allocates a fresh array with the matching elements - it never shares memory with the source.
26	B		Like boolean masking, fancy (integer-array) indexing always returns a copy, never a view.
27	A		reshape() returns a view whenever the data can be reinterpreted from the same contiguous buffer; modifying the reshaped array modifies the original.
28	B		The comma syntax is [row_selector, col_selector]; `` means "all rows", `1` selects column index 1.
29	B		::2` on the row axis steps by 2, selecting every other row; `` on the column axis keeps all

			columns.
30	B		It is the canonical way to verify whether two arrays are views of the same data or fully independent.
31	B		Strided slicing still returns a view; assigning into b writes directly into a's shared memory at the strided positions.
32	B		Pandas inherits the underlying NumPy view/copy ambiguity, which is exactly why .copy() is a defensive habit professionals adopt.
33	B		Rows 0-1 (2 rows) and columns 2-3 (2 columns) → a 2×2 submatrix.
34	B		A step of -1 traverses the array backward from the last to the first element.
35	B		Omitting the stop index is simply shorthand for "go all the way to the end of the array".
36	A		This is the single most common "why isn't my filter sticking" bug. Boolean indexing returns a NEW array - to modify the original conditionally, assign back into it directly, e.g. <code>`data[data["flag"]==1] = 0`</code> , rather than mutating a separately stored filtered copy.

37	B		Element-wise addition between two same-shape arrays adds corresponding positions.
38	B		Verified: even with integer input, <code>np.sqrt()</code> always returns float64 because square roots are generally non-integer.
39	B		<code>exp(0)=1</code> , <code>exp(1)=e$\approx$2.71828</code> .
40	B		Mixed int+float arithmetic always upcasts the result to float to avoid losing fractional precision.
41	B		<code>clip(arr, lo, hi)</code> caps every value to the <code>[lo, hi]</code> range: -5 $\rightarrow$ 0, 3 stays, 15 $\rightarrow$ 10, 8 stays.
42	A		Element-wise absolute value.
43	B		<code>np.log()</code> is natural log (base e); <code>np.log10()</code> and <code>np.log2()</code> are explicit base-10 and base-2 variants. There is no <code>np.ln()</code> .
44	B		<code>floor</code> always rounds down on the number line; <code>trunc</code> simply discards the fractional part, rounding toward zero.
45	B		Verified: <code>-7 // 2 = -4</code> in NumPy (and Python), because floor division always rounds toward negative infinity, unlike

			C-style truncating division.
46	A		np.sign returns -1, 0, or +1 for negative, zero, and positive inputs respectively.
47	B		NumPy follows IEEE 754 floating-point semantics: division by zero produces inf/-inf, and 0/0 produces NaN, with a warning rather than a fatal exception - important for writing crash-resistant pipelines.
48	A		1/0=inf, 2/2=1.0, 0/0=nan - verified directly. Each case follows IEEE 754 rules independently.
49	B		/ always performs floating-point division; // performs floor division.
50	B		Verified equal: the ``*` operator is implemented as a thin wrapper that calls the np.multiply ufunc.
51	B		Ufuncs are the foundation of NumPy's speed - they loop in C, completely bypassing the Python interpreter per element.
52	B		Verified: np.round(2.5)=2.0, np.round(3.5)=4.0. NumPy follows IEEE 754 "round half to even" - a

			common surprise versus the traditional "round half up" rule taught in school.
53	B		A scalar is the simplest broadcasting case: it is logically stretched to every position without copying memory.
54	B		Verified: $\log(0)=-\text{inf}$, $\log(-1)=\text{nan}$, only 1 NaN produced, execution continues. Defensive clipping before <code>log()</code> is standard practice in production analytics code.
55	B		With no axis specified, <code>sum()</code> collapses the ENTIRE array to a single scalar: $0+1+\dots+11 = 66$.
56	B		<code>axis=0</code> collapses the row axis, leaving one aggregated value per column.
57	B		<code>axis=1</code> collapses the column axis, leaving one aggregated value per row.
58	B		Collapsing the 3-row axis leaves a result with one value per column: shape (4,).
59	A		Collapsing the 4-column axis leaves a result with one value per row: shape (3,).

60	B	Verified: for a (3,4) array, axis=-1 produces identical results to axis=1. Negative axis indexing is especially useful for code that must work across arrays of varying dimensionality.
61	B	NumPy defaults to population std (divide by N); pass ddof=1 to get the sample standard deviation (divide by N-1) commonly used in inferential statistics.
62	A	Verified directly: population std for this dataset equals exactly 2.0.
63	B	The "arg" prefix in NumPy functions consistently means "return the index", not the value.
64	B	keepdims=True preserves the reduced dimension as size 1 instead of dropping it entirely - essential for broadcasting the result back against the original array, e.g. when centering data.
65	B	Cumulative sum: 1, 1+2=3, 1+2+3=6, 1+2+3+4=10.
66	B	The 50th percentile is, by definition, the median.

			row.
75	B		Verified: this combination raises ValueError. Broadcasting always compares shapes starting from the TRAILING (rightmost) dimension - a (3,) array aligns against the last axis (size 4 here), not the first.
76	B		Both arrays have a dimension of size 1 in the position where the other has a larger size - both expand, producing the full (3,4) outer-product-style result.
77	B		Refers to the trailing dimension logic.
78	B		Stride-based views are the mechanism that makes broadcasting both memory-efficient and fast.
79	B		mean(axis=0) has shape (4,), broadcasting against each of the 5 rows to subtract the per-column mean.
80	B		Reshaping x to a column (N,1) and a row (1,N) lets broadcasting expand both to (N,N), computing every pairwise difference without an explicit loop.
81	B		Mixed int/float arithmetic always upcasts to the more

			general type, float.
82	B		Verified: shape (8,6). This $(N,1) \times (1,M)$ pattern is the canonical vectorized way to build all-pairs comparison grids.
83	B		Verified: this combination succeeds, producing shape (3,4,5). NumPy pads missing leading dimensions with 1 automatically.
84	B		Verified: incompatible shapes raise ValueError with a descriptive message naming both shapes.
85	B		<code>`*`</code> is ALWAYS element-wise, even when shapes happen to resemble a matrix-vector pattern. Use <code>`@`</code> or <code>np.dot()</code> for true matrix multiplication - confusing these two is the most common NumPy mistake in ML code.
86	B		<code>mean(axis=0)</code> naturally produces shape (4,), which aligns perfectly with the trailing dimension of (50,4) for automatic broadcasting.
87	B		Missing leading dimensions are implicitly treated as size 1 - this is rule #1 of broadcasting.
88	A		A classic trap: many assume any 1D array of

			"the right total length" broadcasts. But broadcasting compares from the TRAILING dimension: (5,3) vs (5,) → (5,3) vs (1,5) → 3≠5 and neither is 1 → ValueError. To add a per-row vector, you need shape (5,1), not (5,).
89	B		The size-1 trailing dimension of (5,1) stretches to match 3, giving (5,3).
90	B		This is exactly the broadcasting pattern that makes per-feature normalization a one-liner across an entire dataset.
91	B		<code>@` (Python 3.5+) is the dedicated matrix multiplication operator, equivalent to <code>np.matmul()/np.dot()</code> for 2D arrays.</code>
92	A		Matrix multiplication: $(m,n) @ (n,p) = (m,p)$. Inner dimensions (4 and 4) must match and cancel out.
93	B		Matrix multiplication requires the inner dimensions to match (A's columns = B's rows). Here $4 \neq 3$, so a transpose is needed first.
94	B		<code>.T</code> is the standard transpose attribute, returning a view with axes swapped.

95	B		Transposing swaps the dimensions: rows become columns and vice versa.
96	A		Verified: matrix-vector multiplication $(m,n) @ (n,) = (m,)$. Each of the 4 rows produces one output value.
97	B		Verified: $A.T$ has shape $(3,4)$; $(3,4) @ (4,3) = (3,3)$. This Gram-matrix pattern appears constantly in linear algebra and ML.
98	A		12 elements split into 2 rows means NumPy infers 6 columns.
99	B		<code>reshape()</code> never adds or drops elements; the total count must match exactly or it raises an error.
100	B		Verified directly: <code>ravel()</code> shares memory with the original, <code>flatten()</code> does not. Choose <code>ravel()</code> when you only need to read the data (faster, no copy); choose <code>flatten()</code> when you need an independent array.
101	B		<code>vstack</code> stacks arrays vertically, adding rows: $2+2=4$ rows, 3 columns unchanged.
102	B		<code>hstack</code> stacks arrays horizontally, adding columns: 2 rows unchanged, $3+3=6$

			columns.
103	B		Verified: the default axis is 0 (stacking along rows), equivalent to vstack for 2D arrays.
104	B		Verified: this raises ValueError. Only the concatenation axis is allowed to differ in size; every other dimension must match exactly.
105	B		By default, np.linalg.norm computes the L2 (Euclidean) length of the vector.
106	B		Normalizing rows to unit vectors makes their dot products directly equal to the cosine of the angle between them - the standard recipe for similarity matrices.
107	B		eye() is the more general/flexible function; identity() is a simpler special case.
108	B		These are fundamentally different operations that happen to share a similar-looking syntax - confusing them is a very common source of subtle bugs in linear-algebra-heavy code.
109	B		Boolean indexing filters and returns the matching VALUES, not their positions.

110	B		Since booleans are a subtype of integers in NumPy, summing a boolean array counts the True values.
111	B		Verified separately: <code>`1 < a < 3`</code> on an array raises "the truth value of an array with more than one element is ambiguous" - exactly the failure that <code>`and`/`or`/chained comparisons</code> trigger on arrays.
112	B		Critical rule: mask on the RIGHT of an assignment = copy (read); mask on the LEFT of an assignment = in-place write.
113	B		This mirrors Python's own <code>sorted()</code> vs <code>list.sort()</code> distinction.
114	B		<code>argsort</code> returns index positions; you then use those indices to reorder the original array or a parallel array.
115	B		<code>argsort</code> gives ascending-order indices; reversing them gives descending order; indexing names with those indices reorders the parallel array correctly.
116	B		Verified: <code>unique()</code> both deduplicates AND sorts the result ascending.

117	B		The two returned arrays have matching positions: counts[i] is the frequency of values[i].
118	B		bincount is faster for its narrow use case (non-negative int frequency counting) but far less general than unique().
119	B		Seeding ensures the exact same "random" sequence is produced every time the code runs - essential for reproducible science and debugging.
120	B		replace=False enforces sampling without replacement, like drawing cards without putting them back.
121	B		shuffle() mutates the input directly, similar to Python's random.shuffle() for lists - use np.random.permutation() if you want a new shuffled copy instead.
122	B		Reproducibility bugs are almost always caused by seed placement, extra random calls consuming state, or environment/version mismatches - not by any inherent randomness in the seeding mechanism itself.
123	B		Verified directly: modifying the fancy-

			indexed result never affects the source array.
124	B		Single-argument <code>np.where()</code> is equivalent to <code>np.nonzero()</code> - it locates positions, not values; for a 1D array, unpack with <code>`[0]`</code> .
125	B		NumPy's convention for <code>argmax/argmin</code> with ties is deterministic - always the first matching position.
126	B		This is one of the most frequent real-world <code>argsort</code> mistakes - always remember <code>argsort</code> defaults to ascending order.
127	B	DEBUG	Plain assignment (<code>`b = a`</code>) never copies array data - it just creates a second name pointing at the same object.
128	B	DEBUG	Verified directly: this raises "Cannot cast ufunc multiply output from dtype float64 to dtype int64 with casting rule same_kind". A common gotcha when slicing-then-modifying an int array with a float operation - fix by using a float dtype array, or <code>`discounted[:] = discounted * 0.5`</code> with explicit casting.
129	B	DEBUG	Verified directly. This is the classic "mutable default argument" trap in